



DiskUrbano

[Início](#)

[Serviços](#)

[Sobre nós](#)

[Contato](#)



Buscar...



Projeto DiskUrbano





Faça sua reclamação

Registre ocorrências urbanas de forma rápida e simples.
Sua cidade melhora com a sua voz.

1 Localização — 2 Tipo — 3 Descrição — 4 Envio



Nova ocorrência

Preencha os dados abaixo para registrar sua reclamação

ESTADO

Selecione...

MUNICÍPIO

Selecione o estado...

TIPO DA OCORRÊNCIA



Buraco na rua



Iluminação quebrada



Semáforo quebrado



Alagamento

Sobre o DiskUrbano

Uma plataforma digital (um protótipo ou interface de serviço público) projetada para que cidadãos possam registrar reclamações e ocorrências urbanas de forma direta. O site pede que o usuário selecione o Estado e o Município, além de fornecer um campo de Descrição detalhada (com limite de caracteres) para que a equipe de manutenção saiba exatamente onde e qual é o problema. O usuário segue um processo de quatro etapas (Localização, Tipo, Descrição e Envio) para reportar um incidente.



package.json

Contém arquivos de configuração para CI/CD (Integração e Entrega Contínua). Automatizar tarefas como testar o código ou fazer o deploy toda vez que algo novo é enviado. Lista todas as bibliotecas externas usadas, versões e scripts de execução (como npm start).

Code

Blame

```
1  {
2    "name": "diskurbano",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1"
8    },
9    "repository": {
10     "type": "git",
11     "url": "git+https://github.com/evedotexe/diskUrbano.git"
12   },
13   "keywords": [],
14   "author": "",
15   "license": "ISC",
16   "type": "module",
17   "bugs": {
18     "url": "https://github.com/evedotexe/diskUrbano/issues"
19   },
20   "homepage": "https://github.com/evedotexe/diskUrbano#readme",
21   "dependencies": {
22     "@prisma/client": "6.19",
23     "dotenv": "^17.3.1",
24     "express": "^5.2.1"
25   },
26   "devDependencies": {
27     "prisma": "6.19"
28   }
29 }
```



schema.prisma

01

Prisma ORM, SQL
(PostgreSQL, MySQL ou
SQLite)

02

Gerencia a comunicação com o banco de dados. Aqui ficam os modelos das tabelas (ex: usuários, ocorrências) e as configurações de conexão. Para salvar dados.

Code

Blame

Raw



```
1 // This is your Prisma schema file,  
2 // learn more about it in the docs: https://pris.ly/d/prisma-schema  
3  
4 // Looking for ways to speed up your queries, or scale easily with your serverless or edge functions?  
5 // Try Prisma Accelerate: https://pris.ly/cli/accelerate-init  
6  
7 generator client {  
8   provider = "prisma-client-js"  
9 }  
10  
11 datasource db {  
12   provider = "mongodb"  
13   url      = env("DATABASE_URL")  
14 }  
15  
16 model ocorrencias {  
17   id        String   @id @default(auto()) @map("_id") @db.ObjectId  
18   estado    String  
19   municipio String  
20   tipo      String  
21   descricao String  
22 }
```

03



Code Biame 46 lines (38 loc) · 1.33 KB

```
1 import "dotenv/config";
2 import express from "express";
3 import cors from "cors";
4 import { PrismaClient } from "@prisma/client";
5 import { fileURLToPath } from 'url';
6 import path from 'path';
7
8 const __filename = fileURLToPath(import.meta.url);
9 const __dirname = path.dirname(__filename);
10
11 const urlPublica = 'https://servicodados.ibge.gov.br/api/v1/localidades/municipios'
12
13 const prisma = new PrismaClient();
14 const app = express();
15
16 app.use(cors());
17 app.use(express.static(path.join(__dirname, 'public')));
18 app.use(express.json());
19
20 app.get('/', (req, res) => {
21   res.sendFile(path.join(__dirname, 'public', 'index.html'));
22 });
23
24 app.get('/municipios', async (req, res) => {
25   const response = await fetch(urlPublica)
26   const data = await response.json()
27   res.json(data)
28 });
29
30 app.post('/ocorrencia', async (req, res) => {
31   try {
32     const { estado, municipio, tipo, descricao } = req.body;
33     const ocorrencia = await prisma.ocorrencias.create({
34       data: { estado, municipio, tipo, descricao }
35     })
36     res.status(201).json({ success: true, message: 'Ocorrência criada com sucesso', data: ocorrencia })
37   }
38   catch (error) {
39     console.error(error)
40     res.status(500).json({ success: false, message: 'Erro ao criar ocorrência' })
41   }
42 })
43
44 app.listen(3000, () => {
45   console.log('servidor rodando :3')
46 })
```

server.js

01

Criação da rota da API, lógica do negócio e conexão com o banco de dados via Prisma.

02

Armazena os arquivos estáticos que o usuário final vê no navegador. Pelo commit "Atualizar index.html", é onde reside o frontend.



src/script

```
Code Blame 40 lines (37 loc) • 1.14 KB

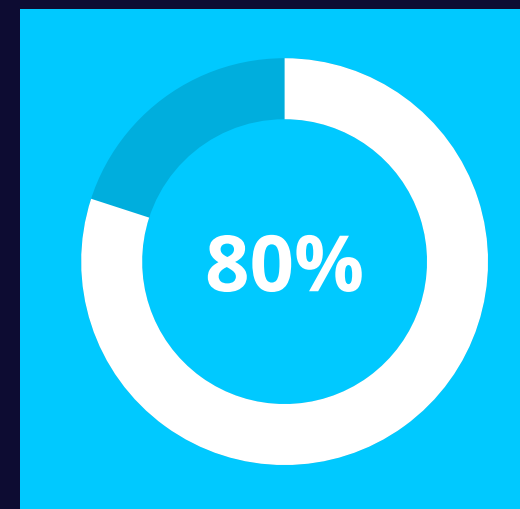
1 export const chamadas = {
2   async buscarCidades() {
3     try{
4       const res = await fetch('http://localhost:3000/municipios')
5       return await res.json()
6     }
7     catch(error) {
8       console.error(error)
9     }
10  },
11
12  async novaOcorrencia(dados) {
13    try {
14      const res = await fetch('http://localhost:3000/ocorrencia', {
15        method: 'POST',
16        headers: {
17          'Content-Type': 'application/json'
18        },
19        body: JSON.stringify({
20          estado: dados.selectEstados,
21          municipio: dados.selectMunicipios,
22          tipo: dados.selectTipo,
23          descricao: dados.descricao
24        })
25      });
26
27      if (res.ok) {
28        alert('Ocorrência registrada com sucesso!');
29        return res.json();
30      } else {
31        throw new Error('Erro ao registrar ocorrência');
32      }
33    }
34    catch(error) {
35      console.error(error);
36      alert('Erro ao registrar: ' + error.message);
37    }
38  }
39
40 }
```

```
Code Blame

1 import { ui } from './ui.js';
2 import { chamadas } from './service/api.js';
3
4 document.addEventListener('DOMContentLoaded', async () => {
5   ui.listarMunicipio();
6
7   document.getElementById('formOcorrencia')
8     .addEventListener('submit', submitEvent);
9 })
10
11 async function submitEvent(event) {
12   event.preventDefault()
13
14   const selectEstados = document.getElementById('inputEstados').value
15   const selectMunicipios = document.getElementById('inputMunicipio').value
16   const selectTipo = document.getElementById('tipoOcorrencia').value
17   const descricao = document.getElementById('descricaoOcorrencia').value
18
19   console.log('Dados do formulário:', { selectEstados, selectMunicipios, selectTipo, descricao })
20
21   if (!selectEstados || !selectMunicipios || !selectTipo || !descricao) {
22     alert('Por favor preencha tudo corretamente!');
23     return;
24   }
25
26   await chamadas.novaOcorrencia({ selectEstados, selectMunicipios, selectTipo, descricao })
27
28   event.target.reset();
29 }
```

01

service/ api.js
Gerencia o fluxo de controle
da aplicação.



02

main.js
É o ponto de entrada do código.
Coordena as ações, o main.js
chama as funções do ui.js para
atualizar a tela e as do service/
para salvar os dados.



.github/workflows

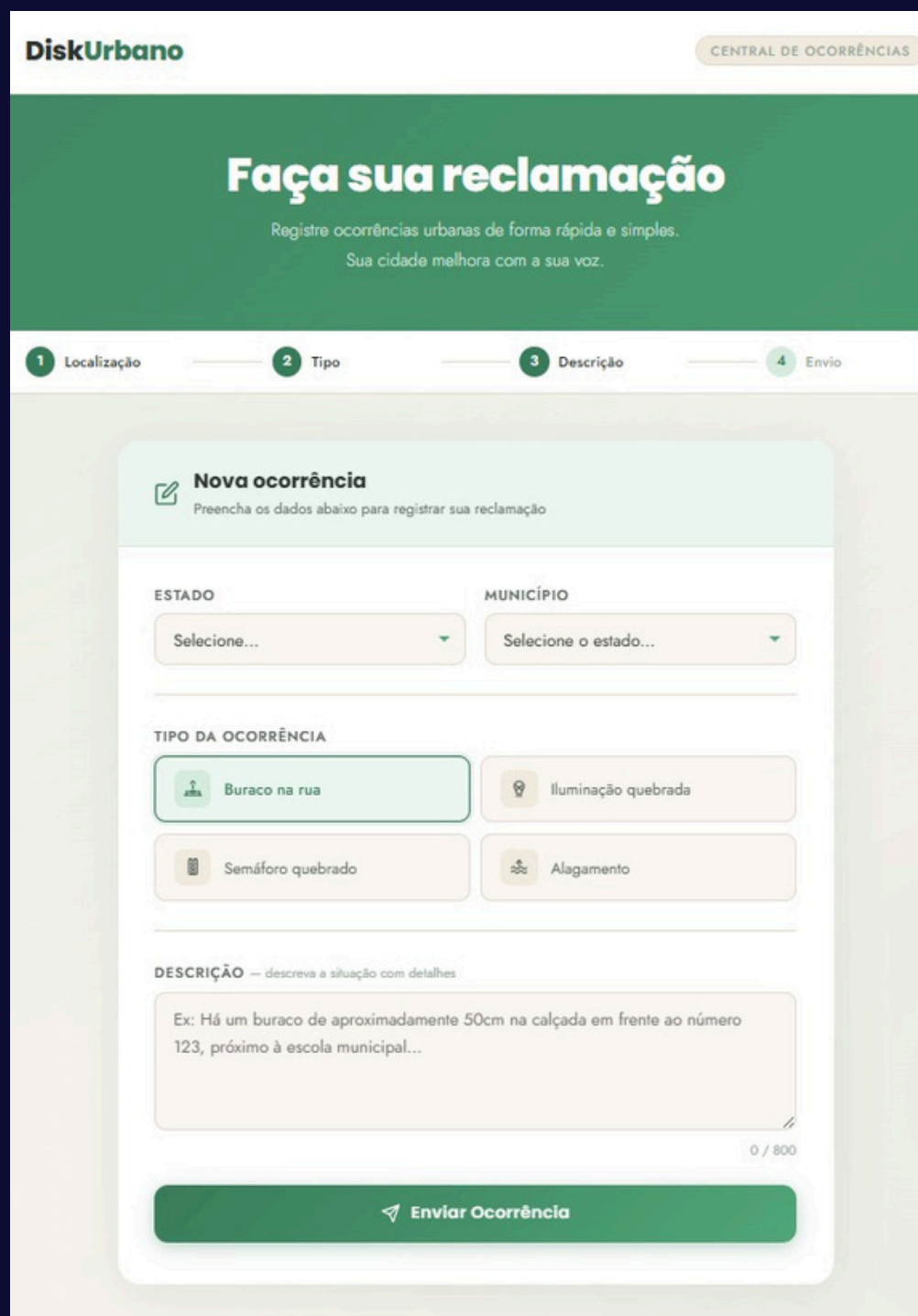
Contém arquivos de configuração para CI/CD (Integração e Entrega Contínua). Configuração de testes automáticos ou deploy automático toda vez que o código é atualizado no GitHub. Ferramentas utilizadas: GitHub Actions e YAML.

```
Code Blame 38 lines (32 loc) · 891 Bytes
1  name: CD
2
3  on:
4    push:
5      branches: [main]
6
7  jobs:
8    build-and-push:
9      runs-on: ubuntu-latest
10     steps:
11       - uses: actions/checkout@v3
12
13       - name: Login Docker Hub
14         uses: docker/login-action@v3
15         with:
16           username: ${{ secrets.DOCKERHUB_USERNAME }}
17           password: ${{ secrets.DOCKERHUB_TOKEN }}
18
19       - name: Build and push image
20         uses: docker/build-push-action@v5
21         with:
22           push: true
23           tags: ${{ secrets.DOCKERHUB_USERNAME }}/diskurbano:latest
24
25     deploy-staging:
26       needs: build-and-push
27       runs-on: ubuntu-latest
28       steps:
29         - name: Deploy staging
30           run: echo "Deploy em staging realizado com sucesso!"
31
32     deploy-production:
33       needs: deploy-staging
34       runs-on: ubuntu-latest
35       environment: production
36       steps:
37         - name: Deploy production
38           run: echo "Deploy em produção aprovado e realizado!"
```

Planejamento da equipe

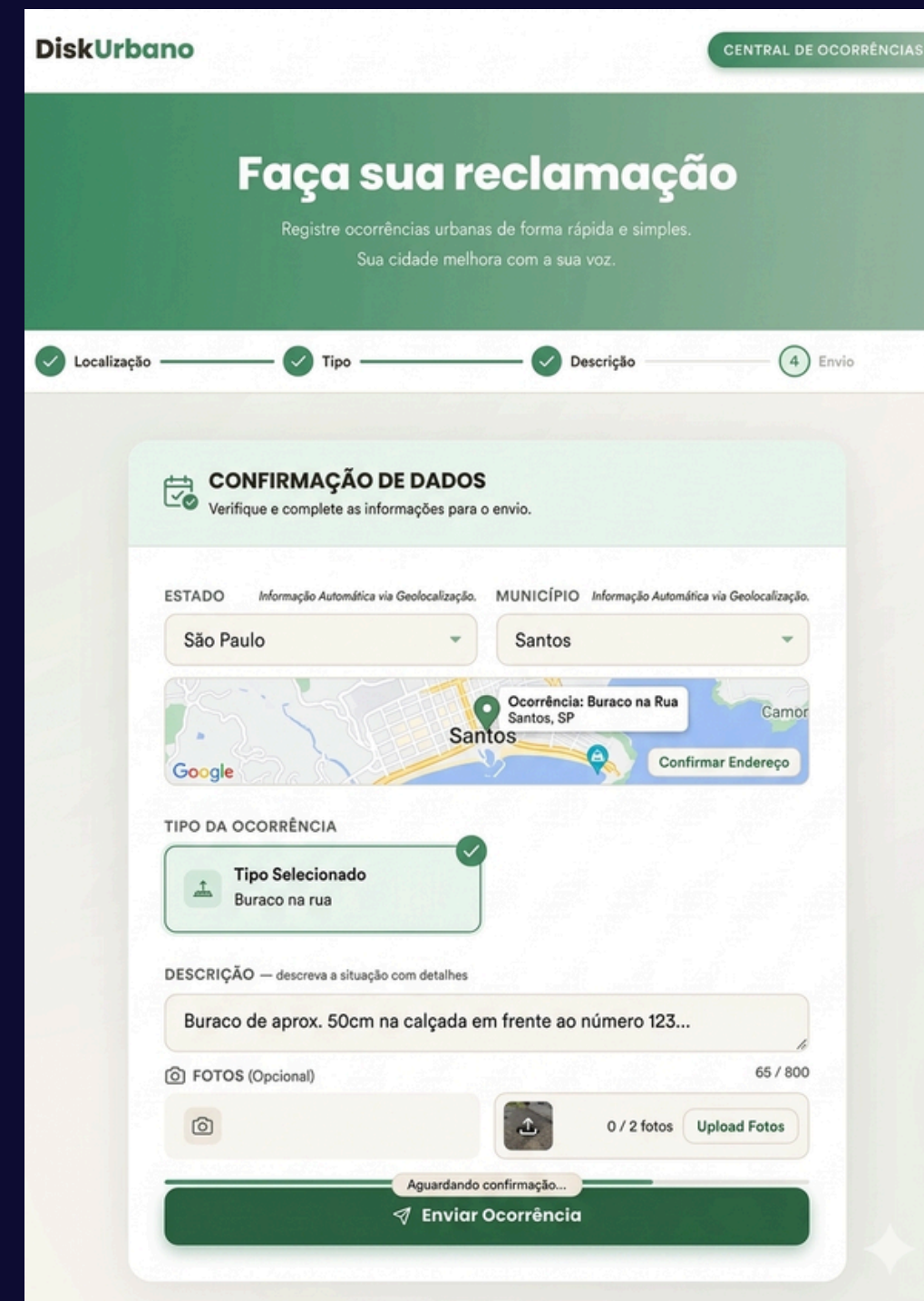
A inclusão e a usabilidade em dispositivos móveis são priorizadas no planejamento do DiskUrbano. Na acessibilidade visual, o objetivo é assegurar um contraste adequado de cores para que todos os cidadãos possam ler textos e rótulos com facilidade.

É fundamental que os títulos dos campos estejam sempre visíveis e que sejam implementados estados de foco nítidos para os usuários que navegam pelo teclado. Em relação à responsividade, organizando os campos de forma vertical em telas pequenas para aproveitar melhor o espaço. Para evitar cliques acidentais, os botões de tipo de ocorrência devem ter áreas de toque grandes. Essas melhorias fazem com que o serviço público digital seja muito mais fácil de usar, rápido e tecnicamente sólido.



The screenshot shows the 'Faça sua reclamação' (Make your complaint) screen in the DiskUrbano app. It features a progress bar at the top with four steps: 1. Localização, 2. Tipo, 3. Descrição, and 4. Envio. The current step is 'Tipo'. The form includes dropdown menus for 'ESTADO' and 'MUNICÍPIO', a section for 'TIPO DA OCORRÊNCIA' with four selectable options (Buraco na rua, Iluminação quebrada, Semáforo quebrado, Alagamento), and a text area for 'DESCRIÇÃO'. A green button at the bottom is labeled 'Enviar Ocorrência'.

INTERFACE ATUAL



The screenshot shows the 'Faça sua reclamação' (Make your complaint) screen in the DiskUrbano app, representing the desired interface. It features a progress bar at the top with four steps: 1. Localização, 2. Tipo, 3. Descrição, and 4. Envio. The current step is 'Tipo'. The form includes dropdown menus for 'ESTADO' and 'MUNICÍPIO', a map section for 'Ocorrência' with a 'Confirmar Endereço' button, a section for 'TIPO DA OCORRÊNCIA' with a 'Tipo Selecionado' indicator, and a text area for 'DESCRIÇÃO'. A green button at the bottom is labeled 'Enviar Ocorrência'.

INTERFACE DESEJADA



Obrigada, Pela atenção!